Testing OAI-UPF with upfbench

Deploy standalone, configure, run all suites — the hows and whys

coRAN Labs

June 2026

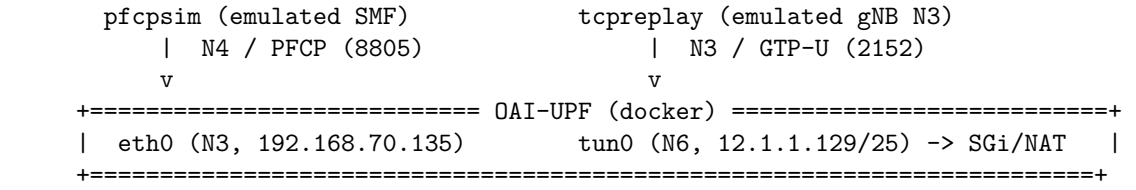
1. What this is and the testing model

OAI-UPF is the **second UPF** integrated into upfbench (after SD-Core BESS-UPF), and the one that proved the framework is genuinely UPF-agnostic. This doc explains how to deploy it **standalone** and benchmark it with all three suites, and — importantly — **why** each non-obvious setting is the way it is.

The testing model is "UPF-isolated". We do **not** drive the UPF through OAI's own SMF. Instead:

- **N4 (control plane)** is driven by **pfcpsim**, which *emulates the SMF* — it associates, then creates/modifies/deletes PFCP sessions directly against the UPF.
- **N3 (user plane)** is driven by **tcpreplay**, which injects GTP-U packets whose TEID/UE-IP match the sessions pfcpsim installed.
- The OAI SMF/AMF are **stopped** during a run so pfcpsim is the *sole* N4 peer.

Why isolate the UPF? Because the goal is to benchmark the **UPF itself** (forwarding rate, session capacity, N4 conformance) without the rest of the core as a variable. This is the same model used for SD-Core; only the adapter and a few knobs differ.



2. How OAI-UPF differs from SD-Core (why it needs its own adapter + knobs)

Aspect	SD-Core BESS-UPF	OAI-UPF
Runs as	Kubernetes pod (`aether-5gc/upf-0`)	**Docker container** (`oai-upf`)
Datapath	BESS (af_packet/AF_XDP/DPDK)	**simpleswitch** (Linux), or VPP/eBPF
N6 egress	BESS `core` port → forwarded == `tx_pkts`	**`tun0` (a TUN device)** → forwarded == `rx_pkts`
Counters read via	`kubectl exec … bessctl show port`	`docker exec … cat /proc/net/dev`
URR IEs	accepted	**rejected** — must be omitted
Association Release	supported	**not implemented**
In-pipeline latency probe	yes (BESS Measure module)	no (black-box) → TC-03/LT-03 skipped
Reset (clean state)	`kubectl delete pod upf-0`	`docker restart oai-upf`

These differences are exactly why upfbench uses a **per-UPF adapter** (`upfbench/adapters/oai_upf.py`) plus a few env-gated pfcpsim knobs — the *suites themselves never change*.

3. Deploy OAI-UPF standalone

OAI doesn't ship a "drive-me-blind" UPF-only image; the practical path is to bring up OAI's **mini 5G core** (the `oai-cn5g-fed` docker-compose: `mysql` + `oai-amf` + `oai-smf` + `oai-upf`, image `oaisoftwarealliance/oai-upf:develop`), then **UPF-isolate** it.

```
# 1. get OAI's core federation + bring up the basic compose
git clone https://gitlab.eurecom.fr/oai/cn5g/oai-cn5g-fed.git
cd oai-cn5g-fed/docker-compose
sudo docker compose -f docker-compose-basic-nrf.yaml up -d   # (compose file name varies by release)

# 2. confirm the UPF is healthy
sudo docker ps | grep oai-upf            # expect: Up … (healthy)

# 3. UPF-ISOLATE: stop OAI's own SMF (and AMF) so pfcpsim owns N4
sudo docker stop oai-smf oai-amf
```

The UPF now sits on the compose's docker bridge (in our reference setup the bridge is **`demo-oai`, 192.168.70.0/26**, UPF at **192.168.70.135**), with **eth0** as the N3 interface and **tun0** (12.1.1.129/25) as the N6/SGi TUN that NATs UE traffic out.

> The framework's reset hook runs `docker restart oai-upf` before each suite, which brings the UPF back with a **fresh, empty session table** — so leave the SMF stopped for the whole run; the reset handles cleanliness.

4. Configure — `configs/oai-upf.yaml`

Two kinds of fields. **Environment-specific** ones MUST match *your* deployment (read them live — don't trust the defaults from another host). **OAI-invariant** knobs encode OAI's quirks and should be left as-is.

4a. Environment-specific (read these off the running container)

Field	What it is	How to read it live
`n4_addr`, `pfcp_remote_addr`	UPF N4 (PFCP) endpoint, `:8805`	`docker inspect -f '{{range .NetworkSettings.Networks}}{{.IPAddress}}{{ oai-upf`
`pfcpsim_iface`, `gen_iface`	host bridge iface on the UPF's docker net	`docker network ls | grep oai`, then `ip -br addr` to find the `br-…`/`demo-oai` iface
`n3_remote_ip`	UPF N3 IP (outer GTP-U dst) — same as the container IP	as above
`ue_pool`	OAI's DNN / SGi subnet (UE IPs)	`docker exec oai-upf ip -br addr` → look at **tun0** (ours 12.1.1.129/25 → pool `12.1.1.0/24`)
`gnb_addr`, `gnb_ip`	source addr the generator uses (outer GTP-U src)	any free addr on the bridge
`inner_dst_ip`	inner packet dst (UE → DN)	a routable DN addr, e.g. `8.8.8.8` (UPF NATs it out)
`gen_cpu_affinity`	pin the generator OFF the UPF datapath cores	keep off cores 0/1; ours `"2-11"`

Field	What it is	How to read it live
`n3_mac_via: arp`	how to resolve the N3 next-hop MAC	`arp` for a docker-bridge UPF (not a k8s pod)

Why this matters: if `ue_pool` doesn't match OAI's actual DNN subnet, the injected uplink packets carry source IPs that match **no PDR**, so the UPF drops them and you see "0 forwarded" even though everything else is healthy.

4b. OAI-invariant knobs (leave as-is — each fixes a real OAI behaviour)

Knob	Why it exists
`pfcpsim_no_urr: true`	OAI-UPF **rejects Usage Reporting Rule (URR) IEs**; including them fails Session Establishment. upfbench doesn't use usage reporting, so we omit them.
`pfcp_no_assoc_release: true`	OAI **doesn't implement graceful PFCP Association Release** (TS 29.244 §7.4.5). The control keeps the association rather than asserting a release it can't do.
adapter `fwd_field() = rx_pkts`	OAI's N6 is **tun0, a TUN device**: the UPF *writes* each decapsulated uplink packet **into** the kernel, which the interface counts as **rx_pkts** (tx stays flat). Verified live. The adapter overrides this — don't change it.
(no Open5GS knobs)	Do **not** set `pfcpsim_apply_action_2b` / `pfcpsim_dnn` / `pfcpsim_fteid_choose` / `pfcpsim_qfi` / `pfcpsim_single_qer`. Those are **Open5GS-only**; OAI accepts the default 1-byte Apply Action and an explicit F-TEID.

TEID/UE-IP **alignment** (the generator crafts GTP-U whose TEID and inner UE IP match the sessions pfcpsim installed) is handled by the framework — you only need `ue_pool` correct.

5. Run it

```
cd ~/Shiva/upf-benchmark-framework

# host mode
sudo -E python3 -m upfbench.cli run --config configs/oai-upf.yaml --suite all --reset-between-suites

# docker mode (identical results)
sudo ./scripts/upfbench-docker.sh run --config configs/oai-upf.yaml --suite all --reset-between-suites
```

- `--reset-between-suites` (also `reset_between_suites: true` in the config) makes the framework `docker restart oai-upf` before each suite. **This is essential for OAI** (see §7).
- For a fast sanity check first: `--suite pfcp` (control-plane only, ~30 s).
- Results land in `campaigns/<campaign-id>/` (`results.json` + `report-*.pdf/.tex`). Use a distinct `campaign:` id if you're comparing cores/modes so reports don't overwrite.

6. What each suite does on OAI, and what to expect

Suite	Cases	Expected on OAI
3 — pfcp	CF-01..05	**5/5 pass.** CF-01 notes "Association Release not supported" — *expected*, not a fail.
2 — load	LT-01/02/03	LT-01 capacity ~**250 sessions** (OAI `max_sessions` limit); LT-02 ~0.028 Mpps aggregate with all verified UEs forwarding; **LT-03 skipped** (no in-pipeline latency probe — that's a BESS-only capability; OAI is black-box).
1 — performance	TC-01/02/03/04/08	TC-01 NDR ~0.011 Mpps; TC-04 burst + TC-08 multi-flow measured; **TC-02 skipped** (needs a real downlink GTP-U path) and **TC-03 skipped** (no in-pipeline latency probe).

Why some tests are "skipped" (not failed): TC-02 (bidirectional) needs a downlink GTP-U encap path we don't synthesize on this rig; TC-03/LT-03 (in-pipeline latency) rely on inserting a measurement module into the datapath, which BESS supports and OAI (black-box) does not. These are honest capability gaps, recorded as `skipped`, not failures.

Absolute numbers are rig-specific (single-NIC VM, simpleswitch, tcpreplay generator ceiling ~0.06 Mpps) and af_packet/simpleswitch throughput is run-to-run noisy. Compare *structure* and *relative* results, not exact figures. If you run OAI-UPF in a **VPP/DPDK** datapath, the UPF can far exceed the tcpreplay generator — then numbers are **generator-limited**, and you'd need **TRex** (the framework has `generator: trex` wired) to find the true ceiling.

7. Troubleshooting / gotchas (learned the hard way)

- **`--suite all` reports LT-01 = 0 / CF-02 fail without reset.** OAI's **batch session establishment wedges after the performance suite's churn** — the perf suite establishes and tears down many sessions + blasts saturating traffic, leaving OAI unable to install the next batch. The next suite (`load`, then `pfcp`) then fails. **Fix:** always use `--reset-between-suites` (the reset hook `docker restart`s OAI before each suite). This is the single most important OAI gotcha and the reason the reset hook exists.
- **"Generator sends but 0 forwarded."** Usually `ue_pool` doesn't match OAI's DNN subnet (uplink matches no PDR), or the UPF wedged — `docker restart oai-upf` and re-run.
- **Establishment times out.** You forgot `pfcpsim_no_urr: true` (OAI rejects URRs), or the OAI SMF is still up and contending on N4 — `docker stop oai-smf`.
- **Wrong IPs.** The `192.168.70.*` / `demo-oai` values are *our* deployment; on a different host read the live container IP + bridge (§4a) and update the config.
- **Run from the repo root** (relative paths) and with `sudo` (tcpreplay/docker).

8. Where this lives in the code

- Adapter: `upfbench/adapters/oai_upf.py` — `docker exec/inspect`, `/proc/net/dev` counters, `fwd_field() -> rx_pkts`, `reset() -> docker restart`.
- Config: `configs/oai-upf.yaml` (the fields above).
- N4 driver: `upfbench/control/pfcpsim.py` (honours `pfcpsim_no_urr`, `pfcp_no_assoc_release`).
- N3 generator: `upfbench/traffic/tcpreplay.py` (crafts the aligned GTP-U pcap with scapy).
- Suites: `upfbench/suites/{performance,load,pfcp}/` (UPF-agnostic; unchanged across UPFs).

See also: `docs/benchmarking-guide.md` (run/reproduce, the reset hook), `docs/docker-deployment.md` (containerized runs), `docs/RUNBOOK.md` (operational detail).